

Scikit-Learn: Complete Theoretical & Mathematical Guide

Table of Contents

1. Data Preprocessing Theory
 2. Classification Algorithms
 3. Regression Algorithms
 4. Unsupervised Learning
 5. Model Evaluation
 6. Cross-Validation & Tuning
-

1. Data Preprocessing Theory

Train-Test Split

When to Use: Always, before any model training

Why: To evaluate model performance on unseen data and detect overfitting

How It Works:

- Randomly divides data into training (typically 70-80%) and testing (20-30%)
- Stratified split maintains class distribution in both sets

Mathematical Concept:

- For dataset D with n samples: $D = D_{\text{train}} \cup D_{\text{test}}$
- $D_{\text{train}} \cap D_{\text{test}} = \emptyset$ (no overlap)

Key Rules:

- Never train on test data
 - Never use test data for any decision-making during training
 - Use stratification for imbalanced datasets
-

Feature Scaling

Standard Scaler (Z-score Normalization)

When to Use:

- Distance-based algorithms (KNN, SVM, K-Means)
- Gradient descent-based algorithms (Linear Regression, Neural Networks)
- When features have different units/scales

Why:

- Prevents features with larger scales from dominating
- Speeds up gradient descent convergence
- Makes features comparable

Formula:

$$z = (x - \mu) / \sigma$$

Where:

- x = original value
- μ = mean of feature
- σ = standard deviation
- z = standardized value

Properties:

- Mean = 0, Standard Deviation = 1
- Preserves outliers

Don't Use When:

- Tree-based algorithms (they're scale-invariant)
 - Features already on same scale
-

MinMax Scaler**When to Use:**

- Need bounded range [0,1] or custom range
- Neural networks (bound activation functions)
- Image data normalization

Why:

- Preserves relationships in data

- Useful when you need specific range

Formula:

$$x_scaled = (x - x_min) / (x_max - x_min)$$

For custom range [a, b]:

$$x_scaled = a + (x - x_min) \times (b - a) / (x_max - x_min)$$

Limitations:

- Sensitive to outliers (they compress the scale)
-

Robust Scaler**When to Use:**

- Data has many outliers
- Need robust statistics

Why:

- Uses median and IQR instead of mean and std
- Resistant to outliers

Formula:

$$x_scaled = (x - median) / IQR$$

Where $IQR = Q3 - Q1$ (75th - 25th percentile)

Encoding Categorical Variables**Label Encoding****When to Use:**

- Ordinal categorical variables (Low, Medium, High)
- Target variable encoding
- Binary categories

Why:

- Converts categories to numbers
- Preserves ordinal relationships

Formula:

Categories: [cat, dog, mouse] $\rightarrow$ [0, 1, 2]

Warning:

- Don't use for nominal variables (creates false ordering)
-

One-Hot Encoding**When to Use:**

- Nominal categorical variables (color, country)
- No natural ordering exists

Why:

- Prevents model from assuming ordinal relationships
- Each category becomes a binary feature

Formula:

For n categories, create n binary columns

Example: Color = [Red, Blue, Green]

Red $\rightarrow$ [1, 0, 0]

Blue $\rightarrow$ [0, 1, 0]

Green $\rightarrow$ [0, 0, 1]

Considerations:

- Creates n columns for n categories
 - Can cause dimensionality explosion
 - Use "drop_first=True" to avoid multicollinearity
-

Feature Selection

SelectKBest

When to Use:

- Too many features (curse of dimensionality)
- Need interpretability
- Reduce training time

Why:

- Removes irrelevant features
- Improves model performance
- Reduces overfitting

How It Works: Uses statistical tests:

- **f_classif**: ANOVA F-statistic for classification
- **chi2**: Chi-squared test (for non-negative features)
- **mutual_info_classif**: Mutual information

Mathematical Concept (F-statistic):

$$F = (\text{Between-group variability}) / (\text{Within-group variability})$$

Higher F → feature better discriminates classes

Recursive Feature Elimination (RFE)

When to Use:

- Need optimal feature subset
- Have trained model available

How It Works:

1. Train model on all features
2. Rank features by importance
3. Remove least important feature
4. Repeat until desired number remains

Why Better Than SelectKBest:

- Considers feature interactions
 - Model-based selection
-

Polynomial Features

When to Use:

- Linear model on non-linear data
- Capture feature interactions

Why:

- Allows linear models to fit curves
- Creates interaction terms

Formula:

For features $[x_1, x_2]$, degree=2:

Original: $[x_1, x_2]$

Polynomial: $[1, x_1, x_2, x_1^2, x_1x_2, x_2^2]$

General: All combinations of features raised to powers $\leq$ degree

Warning:

- Features grow exponentially: n features, degree $d \rightarrow O(n^d)$ features
 - Can cause overfitting
-

2. Classification Algorithms

Logistic Regression

When to Use:

- Binary or multiclass classification
- Need probability estimates
- Linear decision boundary
- Baseline model

Why:

- Fast training and prediction
- Probabilistic interpretation
- Works well with linearly separable data

Mathematics:

For binary classification:

1. Linear combination:

$$z = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n = \mathbf{w} \cdot \mathbf{x} + b$$

2. Sigmoid function:

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

3. Prediction:

$$P(y=1|x) = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$$

4. Decision:

$$\hat{y} = 1 \text{ if } P(y=1|x) \geq 0.5, \text{ else } 0$$

Cost Function (Log Loss):

$$J(\mathbf{w}) = -1/m \sum [y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

Minimized using gradient descent

Key Parameters:

- **C**: Inverse regularization strength (smaller C = stronger regularization)
- **penalty**: 'l1' (sparse), 'l2' (default)

Assumptions:

- Linear relationship between features and log-odds
- Independent observations

Decision Tree Classifier

When to Use:

- Non-linear relationships

- Need interpretability
- Mixed data types (categorical + numerical)
- No feature scaling needed

Why:

- Easy to understand and visualize
- Handles non-linear data naturally
- No preprocessing required

How It Works:

1. Splitting Criterion (Gini Impurity):

$$\text{Gini} = 1 - \sum(p_i^2)$$

Where p_i = probability of class i

Example:

- Pure node [10 class A, 0 class B]: $\text{Gini} = 1 - (1^2 + 0^2) = 0$
- Mixed node [5 A, 5 B]: $\text{Gini} = 1 - (0.5^2 + 0.5^2) = 0.5$

2. Information Gain (Entropy):

$$\text{Entropy} = -\sum(p_i \cdot \log_2(p_i))$$

$$\text{Information Gain} = \text{Entropy}(\text{parent}) - \sum(\text{weighted_entropy}(\text{children}))$$

Algorithm:

1. Start with all data at root
2. For each feature, calculate best split (max information gain)
3. Choose feature with highest gain
4. Recursively split until stopping criterion

Key Parameters:

- **max_depth:** Maximum tree depth (prevents overfitting)
- **min_samples_split:** Minimum samples to split node
- **min_samples_leaf:** Minimum samples in leaf node
- **criterion:** 'gini' or 'entropy'

Pros:

- Interpretable
- No scaling needed
- Handles missing values

Cons:

- Prone to overfitting
 - Unstable (small data changes → different tree)
 - Biased toward features with more levels
-

Random Forest Classifier

When to Use:

- Need high accuracy
- Prevent overfitting of single tree
- Feature importance needed
- Works well "out of the box"

Why:

- Ensemble of trees reduces variance
- More robust than single tree
- Handles high-dimensional data

How It Works (Bagging + Random Subspace):

1. Bootstrap Aggregating:

For each tree:

- Randomly sample n data points WITH replacement
- Train decision tree on this sample

2. Random Feature Selection:

At each split:

- Randomly select k features ($k = \sqrt{n}$ for classification)
- Find best split among these k features only

3. Prediction:

Classification: Majority vote

$$\hat{y} = \text{mode}\{\text{tree}_1(x), \text{tree}_2(x), \dots, \text{tree}_n(x)\}$$

Mathematical Intuition:

Variance of average of N independent models:

$$\text{Var}(\text{average}) = \sigma^2/N$$

Random Forest reduces variance by averaging

Key Parameters:

- **n_estimators**: Number of trees (more = better, but slower)
- **max_features**: Features to consider per split ($\sqrt{n}$ default)
- **max_depth**: Maximum depth of each tree
- **min_samples_split**: Minimum samples to split

Pros:

- Reduced overfitting vs single tree
- Feature importance
- Parallel training

Cons:

- Less interpretable
- Slower than single tree
- More memory intensive

Support Vector Machine (SVM)

When to Use:

- High-dimensional data
- Clear margin of separation exists
- Small to medium datasets
- Need robust decision boundary

Why:

- Finds optimal separating hyperplane
- Works well in high dimensions
- Memory efficient (uses support vectors only)

Mathematics:

Linear SVM:

Goal: Find hyperplane $w \cdot x + b = 0$ that maximizes margin

$$\text{Margin} = 2/\|w\|$$

Optimization:

$$\text{minimize: } (1/2)\|w\|^2$$

$$\text{subject to: } y_i(w \cdot x_i + b) \geq 1 \text{ for all } i$$

Where $y_i \in \{-1, 1\}$

Soft Margin (C parameter):

$$\text{minimize: } (1/2)\|w\|^2 + C \cdot \sum \xi_i$$

Where ξ_i are slack variables allowing misclassification

- Large C: Small margin, fewer misclassifications (hard margin)
- Small C: Large margin, more misclassifications (soft margin)

Kernel Trick (Non-linear SVM):

Transform data to higher dimension where it's linearly separable

Common Kernels:

1. **Linear:** $K(x, x') = x \cdot x'$
 - Use when: Data is linearly separable
2. **RBF (Radial Basis Function):** $K(x, x') = \exp(-\gamma \|x - x'\|^2)$
 - Use when: Non-linear, don't know the structure
 - γ (gamma):
 - Large γ : Tight fit (overfitting risk)
 - Small γ : Loose fit (underfitting risk)
3. **Polynomial:** $K(x, x') = (\gamma \cdot x \cdot x' + r)^d$

- Use when: Features have interactions
- $d = \text{degree}$

Key Parameters:

- **C**: Regularization (1.0 default)
- **kernel**: 'linear', 'rbf', 'poly'
- **gamma**: Kernel coefficient ('scale' = $1/(n_{\text{features}} \times X.\text{var}())$)

Pros:

- Effective in high dimensions
- Memory efficient
- Versatile (different kernels)

Cons:

- Slow on large datasets ($O(n^2)$ to $O(n^3)$)
 - Requires feature scaling
 - Hard to interpret
 - Sensitive to parameters
-

K-Nearest Neighbors (KNN)

When to Use:

- Small to medium datasets
- Non-linear decision boundaries
- Instance-based learning needed

Why:

- Simple, intuitive
- No training phase (lazy learning)
- Naturally handles multiclass

How It Works:

1. For new point x :
 - Calculate distance to all training points
 - Select k nearest neighbors
2. Classification:
 - Majority vote among k neighbors
 - $\hat{y} = \text{mode}\{y_1, y_2, \dots, y_k\}$
3. Weighted voting (optional):
 - Closer neighbors have more influence
 - Weight = $1/\text{distance}$

Distance Metrics:

1. Euclidean (default):

$$d(x, y) = \sqrt{(\sum (x_i - y_i)^2)}$$

2. Manhattan:

$$d(x, y) = \sum |x_i - y_i|$$

Choosing k :

- Small k : Flexible, but noisy (overfitting)
- Large k : Smooth, but may miss local patterns (underfitting)
- Typical: $k = \sqrt{n}$ or use cross-validation
- Use odd k for binary classification (avoid ties)

Key Parameters:

- **n_neighbors**: Number of neighbors (k)
- **weights**: 'uniform' (all equal) or 'distance' (weighted)
- **metric**: Distance function

Pros:

- Simple, no training
- Works well with irregular decision boundaries
- Adapts to new data easily

Cons:

- Slow prediction ($O(n)$ per prediction)
 - Memory intensive (stores all training data)
 - Sensitive to irrelevant features
 - Requires feature scaling
 - Curse of dimensionality
-

Naive Bayes

When to Use:

- Text classification (spam detection, sentiment analysis)
- Fast training needed
- Baseline model
- High-dimensional data

Why:

- Extremely fast
- Works well with small datasets
- Probabilistic predictions

Mathematics (Bayes' Theorem):

$$P(y|x) = P(x|y) \cdot P(y) / P(x)$$

Where:

- $P(y|x)$ = Posterior probability
- $P(x|y)$ = Likelihood
- $P(y)$ = Prior probability
- $P(x)$ = Evidence

Prediction:

$$\hat{y} = \underset{y}{\operatorname{argmax}} P(y) \cdot \prod P(x_i|y)$$

"Naive" Assumption: Features are independent given the class:

$$P(x_1, x_2, \dots, x_n|y) = P(x_1|y) \cdot P(x_2|y) \cdot \dots \cdot P(x_n|y)$$

Types:

1. Gaussian Naive Bayes:

- Use when: Continuous features
- Assumes: Features follow normal distribution

$$P(x_i|y) = (1/\sqrt{2\pi\sigma^2}) \cdot \exp(-(x_i-\mu)^2/(2\sigma^2))$$

2. Multinomial Naive Bayes:

- Use when: Count data (word frequencies, tf-idf)
- Text classification

$$P(x_i|y) = (\text{count}(x_i, y) + \alpha) / (\text{count}(y) + \alpha \cdot n)$$

where α = smoothing parameter

3. Bernoulli Naive Bayes:

- Use when: Binary features
- Document classification (word present/absent)

Pros:

- Very fast training and prediction
- Works well with high-dimensional data
- Requires small training data
- Handles missing data well

Cons:

- Independence assumption rarely holds
- Can't learn feature interactions
- Zero-frequency problem (solved with smoothing)

Gradient Boosting

When to Use:

- Need highest accuracy
- Structured/tabular data

- Willing to tune hyperparameters
- Kaggle competitions

Why:

- Often best performance on tabular data
- Handles mixed data types
- Robust to outliers

How It Works (Boosting):

Unlike Random Forest (parallel), builds trees sequentially:

1. Start with initial prediction:

$$F_0(x) = \underset{\gamma}{\operatorname{argmin}} \sum L(y_i, \gamma) \quad (\text{often mean for regression})$$

2. For $m = 1$ to M :

a. Compute pseudo-residuals:

$$r_{im} = -[\partial L(y_i, F(x_i)) / \partial F(x_i)] \text{ at } F = F_{m-1}$$

b. Fit tree h_m to residuals

c. Update model:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

3. Final prediction:

$$F(x) = F_0(x) + \eta \cdot \sum h_m(x)$$

Intuition: Each tree corrects errors of previous trees

Key Parameters:

- **n_estimators:** Number of boosting stages (more = better, but slower)
- **learning_rate** (η): Shrinkage (0.01-0.1 typical)
 - Small η : Better generalization, needs more trees
 - Large η : Faster training, overfitting risk
- **max_depth:** Tree depth (3-10 typical, shallow trees work well)
- **subsample:** Fraction of samples for each tree (0.8 typical)

Relationship:

$$n_estimators \times learning_rate \approx \text{constant for similar performance}$$

Pros:

- State-of-art accuracy on tabular data
- Handles mixed data types
- Feature importance
- Robust to outliers

Cons:

- Slow training (sequential)
- Sensitive to hyperparameters
- Can overfit easily
- Requires careful tuning

Advanced Versions:

- **XGBoost**: Regularization, parallel processing
 - **LightGBM**: Fast, efficient, handles large data
 - **CatBoost**: Handles categorical features automatically
-

3. Regression Algorithms

Linear Regression**When to Use:**

- Linear relationship exists
- Need interpretability
- Baseline model
- Understand feature impact

Why:

- Simple, fast, interpretable
- Coefficients show feature importance
- Well-understood assumptions

Mathematics:**Model:**

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \varepsilon$$

Matrix form: $y = X\beta + \varepsilon$

Where:

- y = target variable
- X = feature matrix
- β = coefficients
- ε = error term

Ordinary Least Squares (OLS) Solution:

Minimize: $RSS = \sum (y_i - \hat{y}_i)^2$

Closed-form solution:

$$\beta = (X^T X)^{-1} X^T y$$

Interpretation:

β_j = change in y for 1 unit change in x_j (holding others constant)

Assumptions:

1. **Linearity:** Relationship is linear
2. **Independence:** Observations are independent
3. **Homoscedasticity:** Constant variance of errors
4. **Normality:** Errors follow normal distribution
5. **No multicollinearity:** Features aren't highly correlated

Pros:

- Fast, simple
- Interpretable coefficients
- Works well when assumptions hold

Cons:

- Assumes linearity
- Sensitive to outliers
- Fails with multicollinearity
- Can overfit with many features

Ridge Regression (L2 Regularization)

When to Use:

- Multicollinearity exists
- Prevent overfitting
- Many correlated features
- Feature values should shrink but not zero

Why:

- Handles multicollinearity
- Reduces model variance
- All features retained (coefficients shrink)

Mathematics:

Minimize: $RSS + \alpha \cdot \sum \beta_j^2$

$$= \sum (y_i - \hat{y}_i)^2 + \alpha \cdot \|\beta\|_2^2$$

Where α = regularization strength

Closed-form solution:

$$\beta_{\text{ridge}} = (X^T X + \alpha I)^{-1} X^T y$$

Note: $(X^T X + \alpha I)$ is always invertible

Effect of α :

- $\alpha = 0$: Ordinary linear regression
- $\alpha \rightarrow \infty$: $\beta \rightarrow 0$ (model becomes constant)
- Typical: Use cross-validation to find optimal α

Pros:

- Solves multicollinearity
- Prevents overfitting
- Stable solution

Cons:

- All features retained (not sparse)
 - Less interpretable than linear regression
 - Need to choose α
-

Lasso Regression (L1 Regularization)

When to Use:

- Feature selection needed
- Sparse models desired
- Many irrelevant features
- Interpretability important

Why:

- Automatic feature selection
- Creates sparse models (many $\beta = 0$)
- Good for high-dimensional data

Mathematics:

Minimize: $\text{RSS} + \alpha \cdot \sum |\beta_j|$

$$= \sum (y_i - \hat{y}_i)^2 + \alpha \cdot \|\beta\|_1$$

Where α = regularization strength

No closed-form solution (requires iterative methods like coordinate descent)

Feature Selection Property:

As α increases:

- More coefficients become exactly zero
- Effectively performs feature selection

Comparison with Ridge:

Ridge: $\beta_j \rightarrow 0$ but never exactly 0

Lasso: β_j can be exactly 0

Pros:

- Automatic feature selection
- Sparse, interpretable models
- Works well with many features

Cons:

- Arbitrary selection among correlated features
 - Can select at most n features (when $n < p$)
 - Less stable than Ridge with correlated features
-

ElasticNet

When to Use:

- Many correlated features
- Need both regularization and feature selection
- Lasso is too unstable

Why:

- Combines L1 and L2 penalties
- Gets benefits of both Ridge and Lasso
- More stable than Lasso

Mathematics:

Minimize: $RSS + \alpha \cdot [(1-r) \cdot \sum \beta_j^2 + r \cdot \sum |\beta_j|]$

$= RSS + \alpha \cdot [(1-r) \cdot \|\beta\|_2^2 + r \cdot \|\beta\|_1]$

Where:

- α = regularization strength
- r = $l1_ratio$ (mix between L1 and L2)

Parameter Effects:

- $r = 0$: Pure Ridge
- $r = 1$: Pure Lasso
- $0 < r < 1$: ElasticNet

Pros:

- Handles correlated features better than Lasso
- Feature selection like Lasso
- Stability like Ridge

Cons:

- Two hyperparameters to tune
 - More complex than Ridge or Lasso alone
-

Decision Tree & Random Forest Regressor

Same principles as classification, but:

Splitting Criterion:

MSE (Mean Squared Error):

$$\text{MSE} = (1/n) \cdot \sum (y_i - \hat{y})^2$$

Variance Reduction:

Split to minimize weighted MSE of children nodes

Leaf Prediction:

$$\hat{y} = \text{mean}(y \text{ values in leaf})$$

Support Vector Regression (SVR)**When to Use:**

- Non-linear relationships
- Robust to outliers needed
- High-dimensional data

Why:

- Tolerates errors within margin (ϵ -tube)
- Focuses on support vectors

Mathematics:

Minimize: $(1/2)||w||^2 + C \cdot \sum(\xi_i + \xi_i^*)$

Subject to:

- $y_i - (w \cdot x_i + b) \leq \epsilon + \xi_i$
- $(w \cdot x_i + b) - y_i \leq \epsilon + \xi_i^*$

Where:

- ϵ = epsilon (width of tube where errors are ignored)
- ξ_i, ξ_i^* = slack variables
- C = penalty parameter

ϵ -insensitive loss:

Loss = 0 if $|y - \hat{y}| \leq \epsilon$

Loss = $|y - \hat{y}| - \epsilon$ otherwise

Intuition: Only penalize predictions outside the ϵ -tube

Gradient Boosting Regressor

Same as classification, but:

Loss Function:

$L(y, F(x)) = (1/2)(y - F(x))^2$

Pseudo-residuals:

$r_i = y_i - F_{m-1}(x_i)$

Each tree fits to residuals of previous ensemble

4. Unsupervised Learning

K-Means Clustering

When to Use:

- Know number of clusters (k)
- Spherical clusters expected
- Fast clustering needed

Why:

- Simple, fast ($O(nkt)$ where t = iterations)
- Scales well to large datasets

Algorithm:

1. Initialize k centroids randomly
2. Repeat until convergence:
 - a. Assign each point to nearest centroid:
$$c_i = \underset{j}{\operatorname{argmin}} \|x_i - \mu_j\|^2$$
 - b. Update centroids:
$$\mu_j = (1/|C_j|) \cdot \sum(x_i) \text{ for all } x_i \text{ in cluster } j$$
3. Converges when assignments don't change

Objective Function:

$$\text{Minimize: } J = \sum_{j=1} \sum_{i \in C_j} \|x_i - \mu_j\|^2$$

(Within-cluster sum of squares - WCSS)

Choosing k (Elbow Method):

Plot WCSS vs k

Look for "elbow" where WCSS decrease slows

Silhouette Score:

$$s(i) = (b(i) - a(i)) / \max\{a(i), b(i)\}$$

Where:

- $a(i)$ = avg distance to points in same cluster
- $b(i)$ = avg distance to points in nearest cluster

Range: $[-1, 1]$

- Close to 1: Well clustered
- Close to 0: On border
- Negative: Likely wrong cluster

Pros:

- Fast, simple
- Scales to large datasets
- Works well with spherical clusters

Cons:

- Must specify k
 - Sensitive to initialization (use k-means++)
 - Assumes spherical clusters of similar size
 - Sensitive to outliers
-

DBSCAN (Density-Based Spatial Clustering)**When to Use:**

- Don't know number of clusters
- Arbitrary-shaped clusters
- Noise/outliers present
- Clusters have varying densities

Why:

- Finds clusters of arbitrary shape
- Automatically detects outliers
- No need to specify k

Algorithm:

Parameters:

- ϵ (eps): Maximum distance between points
- min_samples: Minimum points to form dense region

Point Types:

1. Core point: $\geq$ min_samples points within ϵ
2. Border point: $<$ min_samples within ϵ , but in neighborhood of core
3. Noise point: Neither core nor border

Algorithm:

1. For each unvisited point p:
 - Find all points within ϵ (ϵ -neighborhood)
 - If $|\text{neighborhood}| \geq \text{min_samples}$:
 - Start new cluster
 - Add all reachable points to cluster
 - Else: mark as noise (may change later)

Density Reachability:

Point q is density-reachable from p if:

- There's chain $p_1, \dots, p_n$ where:
 - $p_1 = p, p_n = q$
 - $p_{i+1} \in N_\epsilon(p_i)$ (ϵ -neighborhood)
 - p_i is core point (except possibly p_n)

Choosing Parameters:

- ϵ : Plot k-distance graph, look for elbow
- min_samples: Use $\geq$ dimensions + 1 (often 4-5 for 2D)

Pros:

- Finds arbitrary-shaped clusters
- Robust to outliers
- No need to specify number of clusters

Cons:

- Struggles with varying density clusters
 - Sensitive to ϵ and min_samples
 - Not fully deterministic (border points)
 - Slow on large datasets
-

Hierarchical Clustering

When to Use:

- Need cluster hierarchy
- Don't know optimal k
- Small to medium datasets
- Want dendrogram visualization

Why:

- No need to specify k initially
- Provides hierarchy (dendrogram)
- Deterministic

Types:

1. Agglomerative (Bottom-up):

1. Start: Each point is a cluster
2. Repeat:
 - Merge two closest clusters
 - Until: One cluster remains

2. Divisive (Top-down):

1. Start: All points in one cluster
2. Repeat:
 - Split cluster
 - Until: Each point is a cluster

Linkage Methods:

1. Single Linkage:

$$d(C_1, C_2) = \min\{d(x, y) : x \in C_1, y \in C_2\}$$

Tends to create long chains

2. Complete Linkage:

$$d(C_1, C_2) = \max \{d(x, y) : x \in C_1, y \in C_2\}$$

Tends to create compact clusters

3. Average Linkage:

$$d(C_1, C_2) = (1/|C_1||C_2|) \cdot \sum_{x \in C_1} \sum_{y \in C_2} d(x, y)$$

4. Ward's Method (Minimizes variance):

Merge clusters that minimize increase in total within-cluster variance
Most commonly used

Pros:

- Hierarchical structure
- No need to specify k upfront
- Deterministic

Cons:

- Computationally expensive $O(n^3)$
- Doesn't scale to large datasets
- Can't undo merges

Principal Component Analysis (PCA)

When to Use:

- Dimensionality reduction
- Visualization (reduce to 2D/3D)
- Remove multicollinearity
- Data compression
- Noise reduction

Why:

- Reduces dimensions while preserving variance

- Removes correlated features
- Speeds up algorithms

Mathematics:

Goal: Find directions (principal components) of maximum variance

1. Standardize data:

$$X_{\text{std}} = (X - \mu) / \sigma$$

2. Compute covariance matrix:

$$C = (1/n) \cdot X^T \cdot X$$

3. Eigendecomposition:

$$C \cdot v = \lambda \cdot v$$

Where:

- v = eigenvectors (principal components)
- λ = eigenvalues (variance explained)

4. Sort eigenvectors by eigenvalue (descending)

5. Select top k eigenvectors: $W = [v_1, v_2, \dots, v_k]$

6. Transform data:

$$X_{\text{pca}} = X \cdot W$$

Variance Explained:

Variance explained by $PC_i = \lambda_i / \sum \lambda_j$

Cumulative variance = $\sum(\lambda_i) / \sum(\lambda_j)$ for $i=1$ to k

Choosing Number of Components:

1. Keep components with cumulative variance $\geq 95\%$
2. Scree plot: Look for elbow
3. Kaiser criterion: Keep components with $\lambda > 1$

Interpretation:

$$PC_1 = w_{11}X_1 + w_{12}X_2 + \dots + w_{1n}X_n$$

Where w_{ij} are loadings (contribution of feature j to PC i)

Pros:

- Removes multicollinearity
- Reduces overfitting
- Speeds up algorithms
- Visualizes high-dimensional data

Cons:

- Components hard to interpret
 - Assumes linear relationships
 - Sensitive to scaling (must standardize)
 - Information loss
-

t-SNE (t-Distributed Stochastic Neighbor Embedding)**When to Use:**

- Visualization only (2D/3D)
- Explore high-dimensional data
- Find patterns, clusters visually

Why:

- Preserves local structure better than PCA
- Reveals clusters and patterns
- Non-linear dimensionality reduction

Mathematics:

Goal: Preserve pairwise similarities in low dimensions

1. High-dimensional similarities (Gaussian):

$$p_{ji} = \exp(-\|x_i - x_j\|^2 / (2\sigma_i^2)) / \sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / (2\sigma_i^2))$$

$$p_{ij} = (p_{ji} + p_{ij}) / (2n)$$

2. Low-dimensional similarities (Student's t):

$$q_{ij} = (1 + \|y_i - y_j\|^2)^{-1} / \sum_{k \neq i} (1 + \|y_i - y_k\|^2)^{-1}$$

3. Minimize KL divergence:

$$\text{Cost} = \text{KL}(P||Q) = \sum_i \sum_j p_{ij} \cdot \log(p_{ij}/q_{ij})$$

Using gradient descent

Why t-distribution?

- Heavy tails → distant points can be farther apart
- Prevents "crowding problem" of high dimensions

Key Parameters:

- **perplexity**: Balance between local and global structure (5-50 typical)
 - Low perplexity: Focus on local structure
 - High perplexity: Focus on global structure
- **n_iter**: Iterations (≥ 1000 recommended)
- **learning_rate**: Step size (10-1000 typical)

Pros:

- Excellent for visualization
- Reveals clusters naturally
- Handles non-linear structures

Cons:

- Only for visualization (can't transform new data)
- Computationally expensive $O(n^2)$
- Different runs give different results
- Distances in t-SNE plot not meaningful
- Perplexity choice affects results

Best Practices:

1. Run multiple times with different random seeds
 2. Try different perplexity values (5, 30, 50)
 3. Use PCA first if > 50 dimensions
 4. Don't over-interpret cluster sizes/distances
-

5. Model Evaluation

Classification Metrics

Confusion Matrix

	Predicted		
	Positive	Negative	
Actual	Pos	TP	FN
	Neg	FP	TN

Where:

- TP = True Positives
 - TN = True Negatives
 - FP = False Positives (Type I error)
 - FN = False Negatives (Type II error)
-

Accuracy

When to Use:

- Balanced datasets
- Equal cost for all errors

Formula:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Proportion of correct predictions

Limitation: Misleading for imbalanced datasets

Example:

Dataset: 95% class A, 5% class B

Model predicting all A: 95% accuracy (but useless!)

Precision

When to Use:

- Cost of False Positives is high
- Spam detection (don't want to mark real emails as spam)
- Medical diagnosis (don't want false alarms)

Formula:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

"Of all predicted positives, how many are actually positive?"

Also called: Positive Predictive Value (PPV)

Interpretation: High precision → Few false alarms

Recall (Sensitivity)

When to Use:

- Cost of False Negatives is high
- Disease screening (don't want to miss sick patients)
- Fraud detection (don't want to miss fraud cases)

Formula:

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

"Of all actual positives, how many did we catch?"

Also called: Sensitivity, True Positive Rate (TPR)

Interpretation: High recall → Few missed positives

F1-Score

When to Use:

- Need balance between precision and recall
- Imbalanced datasets
- Single metric needed

Formula:

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Harmonic mean of precision and recall

Why Harmonic Mean?

- Penalizes extreme values
- F1 is high only if both precision and recall are high

Variants:

$$F\beta = (1 + \beta^2) \times (\text{Precision} \times \text{Recall}) / (\beta^2 \times \text{Precision} + \text{Recall})$$

- $\beta < 1$: Favor precision
- $\beta > 1$: Favor recall
- $\beta = 1$: F1 (balanced)
- $\beta = 2$: F2 (favor recall 2x)

ROC Curve and AUC

When to Use:

- Compare classifiers
- Choose threshold
- Evaluate across all thresholds

ROC Curve (Receiver Operating Characteristic):

X-axis: $FPR = FP / (FP + TN) = 1 - \text{Specificity}$

Y-axis: $TPR = TP / (TP + FN) = \text{Recall}$

Plot TPR vs FPR at various threshold settings

AUC (Area Under Curve):

Range: [0, 1]

- AUC = 1.0: Perfect classifier
- AUC = 0.9-1.0: Excellent
- AUC = 0.8-0.9: Good
- AUC = 0.7-0.8: Fair
- AUC = 0.5: Random classifier (diagonal line)
- AUC < 0.5: Worse than random

Interpretation:

AUC = Probability that classifier ranks random positive higher than random negative

Advantage:

- Threshold-independent
 - Good for imbalanced datasets
-

Regression Metrics

Mean Squared Error (MSE)

Formula:

$$MSE = (1/n) \cdot \sum (y_i - \hat{y}_i)^2$$

Properties:

- Penalizes large errors heavily (squared term)
- Not in original units
- Always positive
- Lower is better

When to Use:

- Want to penalize large errors
 - Optimization (differentiable)
-

Root Mean Squared Error (RMSE)

Formula:

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{[(1/n) \cdot \sum (y_i - \hat{y}_i)^2]}$$

Properties:

- Same units as target variable
- Interpretable
- Sensitive to outliers

When to Use:

- Need interpretable metric in original units
-

Mean Absolute Error (MAE)

Formula:

$$\text{MAE} = (1/n) \cdot \sum |y_i - \hat{y}_i|$$

Properties:

- Same units as target
- Less sensitive to outliers than MSE
- All errors weighted equally

Comparison:

MSE vs MAE:

- MSE: Penalizes large errors more (squared)
- MAE: All errors treated equally (linear)

Choose MAE if outliers should have less influence

R² Score (Coefficient of Determination)

Formula:

$$R^2 = 1 - (SS_{\text{res}} / SS_{\text{tot}})$$

Where:

- $SS_{\text{res}} = \sum (y_i - \hat{y}_i)^2$ (residual sum of squares)
- $SS_{\text{tot}} = \sum (y_i - \bar{y})^2$ (total sum of squares)

Alternative:

$$R^2 = 1 - \text{MSE}(\text{model}) / \text{MSE}(\text{baseline})$$

Interpretation:

$R^2 = 1.0$: Perfect predictions

$R^2 = 0.0$: Model as good as predicting mean

$R^2 < 0.0$: Model worse than baseline (overfitting on train, bad on test)

$R^2 = 0.7$: Model explains 70% of variance

Properties:

- Range: $(-\infty, 1]$
- Not always increasing with more features (use adjusted R^2)

Adjusted R²:

$$R^2_{\text{adj}} = 1 - [(1 - R^2) \cdot (n - 1) / (n - p - 1)]$$

Where:

- n = number of samples
- p = number of features

Penalizes adding features that don't improve fit

Mean Absolute Percentage Error (MAPE)

Formula:

$$\text{MAPE} = (100/n) \cdot \sum |y_i - \hat{y}_i| / |y_i|$$

Properties:

- Percentage $\rightarrow$ scale-independent
- Interpretable
- Undefined when $y_i = 0$

When to Use:

- Compare across different scales
- Need percentage interpretation

Limitation:

- Asymmetric (penalizes under-prediction more)
 - Undefined for zero values
-

6. Cross-Validation & Tuning

Cross-Validation**Why:**

- Get robust performance estimate
 - Use all data for both training and validation
 - Reduce variance in performance estimates
-

K-Fold Cross-Validation**How It Works:**

1. Split data into k equal folds
2. For each fold i:
 - Train on k-1 folds
 - Test on fold i
 - Record performance
3. Average k performance scores

$$\text{Final score} = (1/k) \cdot \sum \text{score}_i$$

Variance:

$$\text{var}(\text{score}) \propto 1/k$$

More folds $\rightarrow$ lower variance, but more computation

Choosing k:

- $k = 5$ or 10 (typical)
- $k = n$: Leave-One-Out (LOOCV) - high variance, expensive
- Larger k : More data per fold, but more computation
- Smaller k : Less computation, but higher variance

Pros:

- All data used for both training and testing
- More reliable than single split
- Less variance than holdout method

Cons:

- Computationally expensive (k model trainings)
 - Not appropriate for time-series (see TimeSeriesSplit)
-

Stratified K-Fold

When to Use:

- Classification with imbalanced classes
- Ensure each fold has same class distribution

How:

Each fold maintains the same class ratio as original dataset

Example: 80% class A, 20% class B

Each fold will also be 80% A, 20% B

Why Better:

- Prevents folds with only one class

- More reliable estimates for imbalanced data
-

Leave-One-Out Cross-Validation (LOOCV)

How:

$k = n$ (number of samples)

Each sample is test set once

n iterations, training on $n-1$ samples each time

Pros:

- Maximum training data ($n-1$)
- Deterministic (no randomness)
- Good for small datasets

Cons:

- Computationally expensive (n iterations)
 - High variance
 - Models highly correlated
-

Hyperparameter Tuning

Grid Search

How It Works:

1. Define parameter grid:

```
param_grid = {  
    'param1': [val1, val2, val3],  
    'param2': [valA, valB]  
}
```

2. Try all combinations:

Total combinations = $|\text{param1}| \times |\text{param2}| \times \dots$

3. For each combination:

- Perform k-fold CV
- Record average score

4. Select best combination

Complexity:

If grid has d dimensions with n values each:

Total models = $n^d \times k$ (folds)

Example: 3 params, 10 values each, 5-fold CV

= $10^3 \times 5 = 5000$ model trainings

Pros:

- Exhaustive search
- Guaranteed to find best combination in grid
- Parallelizable

Cons:

- Exponentially expensive
- Curse of dimensionality
- Might miss optimal values between grid points

Best Practices:

1. Start with coarse grid
 2. Refine around best region
 3. Use domain knowledge for ranges
-

Randomized Search

How It Works:

1. Define parameter distributions:

```
param_dist = {  
    'param1': uniform(0, 10),  
    'param2': randint(1, 100)  
}
```

2. Sample `n_iter` random combinations

3. Evaluate with cross-validation

4. Select best

Advantages over Grid Search:

Probability of finding near-optimal in budget:

Grid: Depends on grid spacing

Random: Depends on `n_iter` (independent of dimensions!)

For same budget:

- Random explores more parameter space
- Better for high dimensions

Mathematical Insight:

For k important parameters out of n :

- Grid: Must have fine spacing in all n dimensions
- Random: Samples effectively in k dimensions regardless of n

Pros:

- Efficient in high dimensions
- Can specify budget (`n_iter`)
- Often finds better results than grid with same budget
- Parallelizable

Cons:

- Not exhaustive
- May miss optimal combination

- Results vary with random seed

When to Use:

- Many hyperparameters (>3)
 - Continuous parameters
 - Limited computational budget
 - Don't know good ranges
-

Model Selection Strategy

Recommended Workflow:

1. Baseline Model:
 - Simple model with default parameters
 - Establishes lower bound
 2. Coarse Random Search:
 - Wide parameter ranges
 - $n_iter = 50-100$
 - Identify promising regions
 3. Fine Grid Search:
 - Narrow ranges around best from step 2
 - More granular grid
 4. Final Validation:
 - Retrain on full training set
 - Test on held-out test set (untouched until now)
-

Bias-Variance Tradeoff

Fundamental Equation:

Expected Error = Bias² + Variance + Irreducible Error

Total Error = $\sum (y_i - \hat{y}_i)^2$

Bias:

$$\text{Bias} = E[\hat{y}] - y_{\text{true}}$$

Error from wrong assumptions in learning algorithm

High Bias $\rightarrow$ Underfitting:

- Model too simple
- Doesn't capture patterns
- Poor on both train and test

Variance:

$$\text{Variance} = E[(\hat{y} - E[\hat{y}])^2]$$

Error from sensitivity to training data fluctuations

High Variance $\rightarrow$ Overfitting:

- Model too complex
- Captures noise
- Great on train, poor on test

Tradeoff:

As model complexity increases:

- Bias decreases (fits training data better)
- Variance increases (more sensitive to training data)

Optimal complexity: Minimize total error

Symptoms and Solutions:

Underfitting (High Bias):

- Train error high, test error high
- Both errors similar

Solutions:

- More complex model
- More features
- Less regularization
- Train longer

Overfitting (High Variance):

- Train error low, test error high
- Large gap between train and test

Solutions:

- Simpler model
- More training data
- Regularization
- Feature selection
- Early stopping
- Cross-validation

Good Fit:

- Both errors low
- Small gap between train and test

Algorithm Selection Guide

Quick Decision Tree

What's your goal?

- └─ Predict category
 - | └─ Interpretability needed → Logistic Regression, Decision Tree
 - | └─ Best accuracy → Random Forest, Gradient Boosting
 - | └─ High dimensions → SVM, Naive Bayes
 - | └─ Text data → Naive Bayes, Logistic Regression
 - | └─ Fast prediction → Naive Bayes, KNN
- └─ Predict number
 - | └─ Linear relationship → Linear, Ridge, Lasso
 - | └─ Feature selection → Lasso, ElasticNet
 - | └─ Multicollinearity → Ridge, ElasticNet
 - | └─ Best accuracy → Random Forest, Gradient Boosting

- └ Find groups
 - └ Know $k \rightarrow$ K-Means
 - └ Don't know $k \rightarrow$ DBSCAN, Hierarchical
 - └ Arbitrary shapes $\rightarrow$ DBSCAN
- └ Reduce dimensions
 - └ Linear $\rightarrow$ PCA
 - └ Visualization $\rightarrow$ t-SNE

Data Size Considerations

Small Data (< 1000 samples):

- Avoid complex models (overfitting risk)
- Use: Naive Bayes, Logistic Regression, Decision Tree
- Careful with: Random Forest, Gradient Boosting
- Essential: Cross-validation

Medium Data (1000 - 100k):

- Most algorithms work well
- Use: Random Forest, SVM, Gradient Boosting
- Tune hyperparameters

Large Data ($> 100k$):

- Avoid: KNN (slow prediction), SVM (slow training)
- Use: Linear models, Random Forest, Gradient Boosting
- Consider: SGDClassifier, MiniBatch algorithms

Feature Characteristics

Categorical Features:

- Tree-based: Handle naturally
- Linear models: Need encoding (one-hot)
- SVM/KNN: Need encoding + scaling

Mixed Types (numerical + categorical):

- Best: Tree-based algorithms
- OK: Linear (with preprocessing)
- Avoid: Naive Bayes (assumes continuous)

High Dimensions ($p > n$):

- Good: Lasso, ElasticNet, Naive Bayes

- OK: SVM with linear kernel
- Avoid: Linear Regression (underdetermined)

Common Pitfalls

1. Data Leakage:

WRONG: Scale before split

```
X_scaled = scaler.fit_transform(X)
```

```
X_train, X_test = split(X_scaled)
```

RIGHT: Split first, then scale

```
X_train, X_test = split(X)
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

2. Using Test Set Multiple Times:

Test set should be touched only once for final evaluation

Use validation set or cross-validation for tuning

3. Ignoring Class Imbalance:

Solutions:

- Stratified split
- Class weights
- Over/under sampling (SMOTE)
- Appropriate metrics (F1, not accuracy)

4. Not Scaling:

Always scale for: SVM, KNN, Linear models, Neural Networks

Never needed for: Tree-based algorithms

5. Overfitting to Validation Set:

Too many tuning iterations → overfitting to validation

Solution: Hold out final test set completely

Final Recommendations

Start Simple:

1. Baseline model (mean, mode, or simple model)
2. Linear model (Logistic/Linear Regression)
3. If needed, increase complexity

Always:

- Split your data properly
- Use cross-validation
- Scale when needed
- Check for overfitting
- Use appropriate metrics

Model Complexity Ladder:

Simple → Complex:

Linear → Polynomial → Tree → Random Forest → Gradient Boosting

Start simple, increase only if needed

Remember:

"Better data beats better algorithms"

Focus on:

- Quality features
- Proper preprocessing
- Understanding your problem

Before:

- Complex models
- Extensive tuning